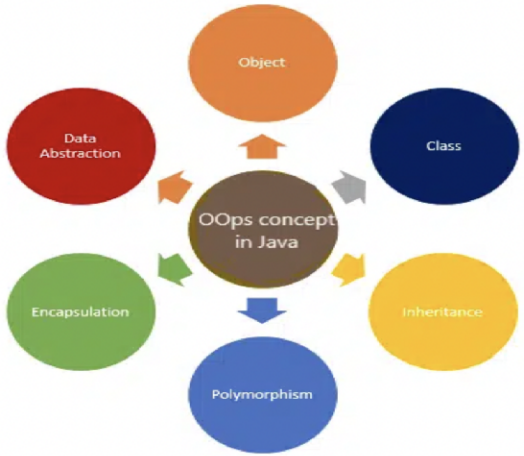
Object Oriented Programming In Java

Saurabh Pandey
Associate Professor
Sitare University

OOPS Core Concepts

- Class
- Object
- Data Abstraction
- Encapsulation
- Inheritance
- Polymorphism



Inheritance

Inheritance is the process of one class inheriting properties and methods from another class in Java. Inheritance is used when we have **is-a** relationship between objects. Inheritance in Java is implemented using **extends** keyword. It avoids the duplicate code.

What is has-a relationship?

There are 5 different types of inheritance as follows:

- **Single Inheritance:** Class B inherits Class A using extends keyword
- **Multilevel Inheritance:** Class C inherits class B and B inherits class A using extends keyword
- **Hierarchy Inheritance:** Class B and C inherits class A in hierarchy order using extends keyword
- **Multiple Inheritance:** Class C inherits Class A and B. Here A and B both are superclass and C is only one child class. **Java is not supporting Multiple Inheritance**, but we can implement using Interfaces.
- **Hybrid Inheritance:** Class D inherits class B and class C. Class B and C inherits A. Here same again Class D inherits two superclass, so **Java is not supporting Hybrid Inheritance** as well.

Example

```java
// super class
class Car {
    // the Car class have one field
    public String wheelStatus;
    public int noOfWheels;

    // the Car class has one constructor
    public Car(String wheelStatus, int noOfWheels)
    {
        this.wheelStatus = wheelStatus;
        this.noOfWheels = noOfWheels;
    }

    // the Car class has three methods
    public void applyBrake()
    {
        wheelStatus = "Stop";
          System.out.println("Stop the car using break");
    }

    // toString() method to print info of Car
    public String toString()
    {
        return ("No of wheels in car " + noOfWheels + "\n"
                + "status of the wheels " + wheelStatus);
    }
}
```

```java
// sub class
class Honda extends Car {

    // the Honda subclass adds one more field
    public Boolean alloyWheel;

    // the Honda subclass has one constructor
    public Honda(String wheelStatus, int noOfWheels,
                 Boolean alloyWheel)
    {
        // invoking super-class(Car) constructor
        super(wheelStatus, noOfWheels);
        this.alloyWheel = alloyWheel;
    }

    // the Honda subclass adds one more method
    public void setAlloyWheel(Boolean alloyWheel)
    {
        this.alloyWheel = alloyWheel;
    }

    // overriding toString() method of Car to print more
    // info
    @Override public String toString()
    {
        return (super.toString() + "\nCar alloy wheel "
                + alloyWheel);
    }
}
```

```java
// driver class
public class Main {
    public static void main(String args[])
    {

        Honda honda = new Honda("stop", 4, true);
        System.out.println(honda.toString());
    }
}
```

Output :
No of wheels in car 4
Status of the wheels stop
Car alloy wheel true

Inheritance and Access modifiers

- Public members are inherited.
- Private members are not inherited.
- default members are inherited in the same package subclass but not in different package subclass.
- Protected members are inherited in the different package subclass too.

Visibility	Public	Protected	*Default*	Private
From the same class	Yes	Yes	Yes	Yes
From any class in the same package	Yes	Yes	Yes	No
From a subclass in the same package	Yes	Yes	Yes	No
From a subclass outside the same package	Yes	Yes, *through inheritance*	No	No
From any non-subclass class outside the package	Yes	No	No	No

Only super class methods can be overridden not the instance variables. Only those methods should be overridden whose behaviors may change in subclasses.

If subclass wants to use super class method, use super.methodname() in the subclass method.

Restrict Inheritance

- If you declare a class as final, no other class can extend it. Java APIs has many classes as final. Example : String Class
- If you declare an instance variable as final, you can’t change it’s value.
- If you declare a method as final, subclass can’t override it.

Rules for Overriding:

- Arguments must be the same and return type must be compatible.
- The method can’t be less accessible.

Overloading a method

Method overloading is nothing more than having two methods with the same name but different argument lists. It has nothing to do with inheritance and polymorphism. An overloaded method is NOT the same as an overridden method.

Rules of Overloading:

1. The return types can be different.
2. You can’t change ONLY the return type.
3. You can vary the access levels in any direction.

Abstraction

Abstraction is a process of hiding implementation details and exposing only the functionality to the user. In abstraction, we deal with ideas and not events. This means the user will only know “what it does” rather than “how it does”.

There are two ways to achieve abstraction in Java:

- Abstract class (0 to 100%)
- Interface (100%)

Certain key points should be remembered regarding this pillar of OOPS as follows:

- The class should be marked as abstract if a class has one or many abstract methods.
- An abstract class can have constructors, concrete methods, static method, and final method
- Abstract class can’t be instantiated directly with the ***new*** operator. It can be possible as shown in pre tag below:

A obj = new B();

//Here A is abstract class and B is the concrete sub class of A.

- The child class should override all the abstract methods of parent class the child class should be declared with

Example:

```java
// Abstract class
public abstract class Car {
    public abstract void stop();
}

// Concrete class
public class Honda extends Car {
    // Hiding implementation details
    @Override public void stop()
    {
        System.out.println("Honda::Stop");
        System.out.println(
            "Mechanism to stop the car using break");
    }
}

public class Main {
    public static void main(String args[])
    {
        Car obj
            = new Honda(); // Car object =>contents of Honda
        obj.stop(); // call the method
    }
}
```

Output:
Honda::Stop
Mechanism to stop the car using break

Abstract Class

- The compiler won’t let you instantiate an abstract class
- An abstract class has virtually* no use, no value, no purpose in life, unless it is extended
- An abstract method has no body!
- If you declare an abstract method, you MUST mark the class abstract as well. You can’t have an abstract method in a non-abstract class.
- You MUST implement all abstract methods in the first concrete class. Implementing an abstract method is just like overriding a method.

Interface

- A Java interface is like a 100% pure abstract class.
- Define an interface
- Implement an interface
- Classes from different inheritance trees can implement the same interface.
- A class that implements an interface must implement all the methods of the interface, since all interface methods are implicitly public and abstract
- Default methods
- Static methods
- Private methods

Object Class important methods

- equals(Object o)
- getClass()
- hashCode()
- toString()

Polymorphism

Polymorphism refers to many forms, or it is a process that performs a single action in different ways. It occurs when we have many classes related to each other by inheritance.

There are two types of polymorphism as listed below:

1. Static or Compile-time Polymorphism
2. Dynamic or Run-time Polymorphism

Static or Compile-time Polymorphism when the compiler is able to determine the actual function, it's called **compile-time** polymorphism. Compile-time polymorphism can be achieved by **method overloading** in java. When different functions in a class have the same name but different signatures, it's called method overloading. A method signature contains the name and method arguments. So, overloaded

```java
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }
    public double add(double a, double b) {
        return a + b;
    }
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        int sum1 = calc.add(5, 10);
        System.out.println("Sum of 5 and 10 (integers): " + sum1);
        double sum2 = calc.add(2.5, 3.7);
        System.out.println("Sum of 2.5 and 3.7 (doubles): " + sum2);
    }
}
```

Compile Time or Static

```
Sum of 5 and 10 (integers): 15
Sum of 2.5 and 3.7 (doubles): 6.2
```

Polymorphism

Dynamic or Run-time Polymorphism occurs when the compiler is not able to determine whether it's superclass method or sub-class method it's called **run-time** polymorphism. The run-time polymorphism is achieved by **method overriding**. When the superclass method is overridden in the subclass, it's called method overriding.

```
class Bike{
  void run(){System.out.println("running");}
}
class Splendor extends Bike{
  void run(){System.out.println("running safely with 60km");}

  public static void main(String args[]){
    Bike b = new Splendor();//upcasting
    b.run();
  }
}
```

```
running safely with 60km.
```